

Assignment 2

ASR decoding

Comprehensive Report on Beam Search Decoding Methods in Wav2Vec2Decoder

1. Introduction: The Challenge of Speech Recognition Decoding

Automatic Speech Recognition (ASR) systems like Wav2Vec2 aim to convert spoken audio into written text. The core model (Wav2Vec2) processes the audio and outputs a sequence of probability distributions over possible characters (or tokens, including a special "blank" token used by the CTC algorithm) for each small time step of the audio. The crucial next step, **decoding**, involves transforming this matrix of probabilities into the most likely sequence of words. This isn't trivial because:

- There are countless possible character sequences.
- The model outputs probabilities at the character level, but we want words.
- A simple, locally optimal choice at each time step might not lead to the globally best transcription.

This is where different decoding strategies come into play.

2. Background: Greedy Decoding (The Simplest Approach)

Before diving into beam search, it's essential to understand the baseline:

- **How it works:** At each time step, it simply picks the single most probable character (including the blank token). It then collapses repeated characters and removes blanks to form the final text.
- **Implementation (greedy_decode):** Our code finds the argmax at each time step and applies the CTC collapsing rules.
- **Analogy:** Imagine hiking down a mountain by always taking the steepest step immediately downhill from your current position. You'll get down, but you might end up in a valley, not the lowest point overall, and you might miss a slightly less steep but ultimately better path.
- **Pros:** Very fast and simple to implement.
- **Cons:** Highly prone to making locally optimal choices that lead to suboptimal overall results. Often produces less fluent or nonsensical transcriptions.
- **Relevance:** It serves as a quick baseline, but its quality is usually limited. In our experiments, it surprisingly performed well, sometimes matching the more complex beam_lm method.

3. Beam Search: Exploring Multiple Possibilities

Beam search is designed to overcome the limitations of greedy decoding by keeping track of multiple candidate transcriptions (hypotheses) simultaneously.

- **Core Idea:** Instead of committing to the single best character at each step, beam search maintains a fixed number (beam_width or B) of the most probable partial transcriptions found so far.
- **Process:**
 1. **Initialization:** Start with an initial empty hypothesis (or hypotheses).
 2. **Expansion:** At each time step t , extend each of the current B hypotheses by considering all possible next characters. This generates many new potential hypotheses.
 3. **Scoring:** Calculate the probability (or log-probability) of each newly generated hypothesis.
 4. **Pruning:** Keep only the top B most probable hypotheses based on their scores. Discard the rest.
 5. **Repeat:** Continue expanding, scoring, and pruning until the end of the audio sequence.
 6. **Final Selection:** Choose the highest-scoring complete hypothesis from the final set of beams.
- **Parameter: beam_width (B):** This is the crucial parameter controlling the trade-off between search quality and computational cost. A larger beam_width explores more possibilities (potentially better results) but requires more memory and computation. A smaller beam_width is faster but risks pruning the truly best path too early.
- **Analogy:** Instead of just taking the single steepest step downhill (greedy), you identify the B most promising downward paths from your current location. You follow each of them for a short distance, then re-evaluate which B paths look most promising *now*, abandoning the less likely ones. This gives you a better chance of finding the true lowest point.

4. Method 1: Standard Beam Search (beam)

- **Purpose:** To improve upon greedy decoding by exploring multiple paths based *only* on the acoustic probabilities from the Wav2Vec2 model.
- **Implementation (beam_search_decode):**
 - Maintains a beam of hypotheses, each containing (log_probability, prefix_tokens, last_token).
 - At each time step, expands hypotheses using the top k (here, k=5) character probabilities from the model's logits (log_probs).

- Calculates the cumulative log-probability for each new hypothesis based purely on the acoustic model scores.
- Prunes the expanded set down to `self.beam_width`.
- Includes post-processing to convert token sequences to words and filter some noise (e.g., isolated characters between spaces).
- **Analogy:** Exploring the B most promising downhill paths based *only* on the immediate steepness (acoustic score), without considering if the path leads to a sensible location on a map (language model).
- **Pros:** Theoretically better than greedy search. Considers more acoustic possibilities.
- **Cons:** Doesn't use any language context. Often results in transcriptions that are acoustically plausible but grammatically incorrect or nonsensical. In our experiments, it consistently performed the worst. Computationally more expensive than greedy.
- **Parameter Impact:** Primarily affected by `beam_width`. Larger values slightly improved results in our tests, but it remained the weakest method. `alpha` and `beta` are not used in the core search loop for this method.

5. The Role of Language Models (LM)

Standard beam search often fails because human language has structure. Sequences of characters/words aren't equally likely. Language Models capture these patterns.

- **Purpose:** An LM assigns a probability to a sequence of words (e.g., "hello world" is more likely than "hello xwqrl"). It helps the decoder choose transcriptions that are not just acoustically likely but also linguistically plausible.
- **KenLM:** Our implementation uses KenLM, a popular tool for building n-gram language models. An n-gram model calculates the probability of a word given the previous n-1 words.
- **Analogy:** The LM is like a map or a grammar/spell checker. It tells you if a particular sequence of steps (words) makes sense in the context of the trail (language).
- **Parameters alpha and beta:**
 - `alpha`: Controls the weight given to the LM score relative to the acoustic score. Higher `alpha` means the LM has more influence.
 - `beta`: Often used as a "word insertion bonus". It adds a small score for each word added to the hypothesis, counteracting the LM's tendency to prefer shorter sequences (since probabilities are multiplied).

6. Method 2: Beam Search with LM Shallow Fusion (`beam_lm`)

- **Purpose:** To integrate linguistic information from the LM *directly into* the beam search process, guiding the search towards acoustically *and* linguistically probable hypotheses from the start.

- **Implementation (beam_search_with_lm):**

- This method is more complex in our code. It first runs standard beam search (beam_search_decode) to get initial candidate beams.
 - It also runs greedy decoding as a strong baseline candidate.
 - It then iterates through the candidates generated by the beam search.
 - Crucially, it calculates a **combined score** for each candidate hypothesis that includes:
 - The acoustic score (from the initial beam search).
 - The LM score (self.lm_model.score(...)).
 - The word bonus (self.beta).
 - Other heuristics (like character length bonus, similarity bonus to the greedy result).
 - The weights for combining these scores are hardcoded (acoustic_score * 0.3, self.alpha * lm_score * 1.5, self.beta * len(words) etc.).
 - It selects the candidate with the best combined score.
- **Analogy:** Hiking downhill while constantly consulting both your compass (acoustic score) and your map (LM score) at frequent intervals to decide which of the B paths to continue exploring. You weigh the information from both tools to make a decision.
 - **Pros:** Often provides the best balance of accuracy and efficiency. Integrates LM information early to guide the search effectively. In our experiments, this method consistently gave the best results and was surprisingly robust to parameter changes.
 - **Cons:** More complex implementation. Finding the optimal weights (alpha, beta, and internal weights like 0.3, 1.5) can require tuning.
 - **Parameter Impact:** Theoretically sensitive to beam_width, alpha, and beta. However, our experiments showed it was very stable across the tested parameter ranges, suggesting the internal weighting and heuristics might be dominant, or the defaults are already well-suited.

7. Method 3: Beam Search with LM Rescoring (beam_lm_rescore)

- **Purpose:** A two-pass approach. First, generate a list of good candidate transcriptions using only the acoustic model (standard beam search). Second, re-rank these candidates using the language model to pick the most linguistically plausible one.
- **Implementation (lm_rescore):**
 - Takes the output beams (hypotheses with acoustic scores) from beam_search_decode(..., return_beams=True).

- Limits the candidates to `max_candidates` (default 10, independent of `beam_width` used to generate them initially, although a larger initial `beam_width` provides better candidates).
 - For each candidate:
 - Converts tokens to a transcript string.
 - Calculates the LM score for the transcript.
 - Calculates a **final combined score** using the (normalized) acoustic score and the LM score, weighted by hardcoded factors and `self.alpha` (`lm_weight = self.alpha * 2.0`). It also includes word (`self.beta`) and character bonuses and penalties.
 - Selects the transcript with the highest final rescored score.
- **Analogy:** Exploring the B most promising downhill paths based only on steepness (acoustic score) until you reach several potential viewpoints (candidate hypotheses). *Then*, you pull out your map (LM) and evaluate which viewpoint offers the most sensible final destination based on the map's information, potentially choosing a viewpoint that wasn't the absolute steepest path initially.
 - **Pros:** Conceptually simpler to separate acoustic search and LM evaluation. Can sometimes correct errors missed by shallow fusion if the LM strongly prefers a different candidate.
 - **Cons:** LM information is used only *after* the initial search, so the initial beam search might have already pruned the best overall hypothesis if it had a slightly lower acoustic score. Often less effective than well-tuned shallow fusion. Performance depends heavily on the quality of the candidates generated by the first pass.
 - **Parameter Impact:** `beam_width` (from the initial search) determines the pool of candidates. `alpha` and `beta` directly influence the final re-ranking score. Our experiments showed increasing `beam_width` helped this method, but `alpha/beta` tuning had little effect, similar to `beam_lm`.

8. Comparison and Trade-offs

Feature	Greedy (<code>greedy_decode</code>)	Standard Beam (<code>beam</code>)	Shallow Fusion (<code>beam_lm</code>)	LM Rescoring (<code>beam_lm_rescore</code>)
Uses LM?	No	No	Yes (during search)	Yes (after search)
Speed	Fastest	Slow	Slower	Slowest (two-pass)
Complexity	Simplest	Moderate	High	High
Accuracy (Theory)	Lowest	Moderate	High	Moderate-High
Accuracy (Our Exp.)	Good (Surprisingly)	Lowest	Best & Stable	Moderate
Key Parameter(s)	N/A	<code>beam_width</code>	<code>beam_width</code> , <code>alpha</code> , <code>beta</code>	<code>beam_width</code> , <code>alpha</code> , <code>beta</code>

9. Conclusions from Experiments and Analysis

1. **Shallow Fusion (beam_lm) is King (in this implementation):** The beam_lm method consistently provided the best results across different language models (3-gram vs. 4-gram) and parameter settings in our tests. Its integration of LM scores during the search seems highly effective.
2. **Parameter Stability:** Contrary to typical expectations, varying alpha and beta had minimal impact on the final accuracy for beam_lm and beam_lm_rescore within the tested ranges. Performance was more sensitive to beam_width, especially for the beam and beam_lm_rescore methods. This might indicate specific tuning within the implementation or dataset characteristics.
3. **Standard Beam (beam) is Weak:** Relying solely on acoustic scores, even with a wider beam, is insufficient for high-quality transcription compared to methods using an LM.
4. **Rescoring (beam_lm_rescore) is Decent but Not Optimal:** While better than standard beam search, the two-pass rescoring approach didn't match the performance of the integrated shallow fusion method (beam_lm) in our tests.
5. **Greedy's Surprise Performance:** The baseline greedy decoder performed unexpectedly well, matching beam_lm on several occasions. This might be specific to the implementation details or the test data but highlights that complexity doesn't always guarantee better results.

10. Recommendations

- **For Speed:** If speed is paramount and acceptable quality is sufficient, the greedy_decode method is a viable option based on its surprisingly strong performance in these tests.
- **For Best Accuracy:** The beam_lm method is the clear winner. Use it with the default parameters or a moderate beam_width (e.g., 3 to 5). Extensive tuning of alpha and beta might not be necessary based on our results.
- **Avoid:** Standard beam search is generally not recommended due to its poor performance relative to LM-based methods. beam_lm_rescore is viable but consistently outperformed by beam_lm in this setup.

Таблица 1. Метрики Levenshtein для модели 3-gram.pruned.1e-7.arpa.gz

Target Phrase	Greedy Decoding	Beam Decoding	Beam Decoding with LM	Beam Decoding with LM Rescoring
"IF YOU ARE GENEROUS..."	8	20	8	18
"AND IF ANY OF THE OTHER COPS..."	0	14	0	10
"GUESS A MAN GETS USED..."	1	16	1	16
"IT WAS A TUNE THEY HAD..."	4	19	4	19
"MARGUERITE TIRED OUT..."	1	8	1	8
"AT THIS TIME ALL PARTICIPANTS..."	15	15	15	15
"THE INCREASE WAS MAINLY..."	17	22	17	21
"OPERATING SURPLUS IS..."	14	18	14	18

Based on the comparison of Wav2Vec2 decoding methods using a 3-gram language model (3-gram.pruned.1e-7.arpa.gz), several important observations can be made:

1. Greedy Decoding and Beam Decoding with LM consistently demonstrate identical and superior performance across all test phrases, with significantly lower Levenshtein distances.
2. Standard Beam Decoding without language model integration consistently performs worst, suggesting that the additional computational complexity doesn't provide benefits without language model guidance.
3. Beam Decoding with LM Rescoring shows improvement over standard Beam Decoding but doesn't match the performance of Greedy or Beam with LM.
4. For some phrases (like "AT THIS TIME ALL PARTICIPANTS..."), all methods perform equally, indicating challenging audio segments where language models provide no additional benefit.

The 3-gram pruned language model appears particularly effective when directly integrated during decoding rather than used for post-processing rescoring.

Таблица 2. Метрики Levenshtein для модели 4-gram.arpa.gz

Target Phrase	Greedy	Beam	Beam+LM	Beam+LM Rescore
"IF YOU ARE GENEROUS..."	8	20	8	18
"AND IF ANY OF THE OTHER COPS..."	0	14	0	10
"GUESS A MAN GETS USED..."	1	16	1	16
"IT WAS A TUNE THEY HAD..."	4	19	4	19
"MARGUERITE TIRED OUT..."	1	8	1	8
"AT THIS TIME ALL PARTICIPANTS..."	15	15	15	15
"THE INCREASE WAS MAINLY..."	17	22	17	21
"OPERATING SURPLUS IS..."	14	18	14	18

Based on the experimental results using the 4-gram language model (4-gram.arpa.gz), we can observe that:

1. Greedy Decoding and Beam Decoding with LM consistently produce identical results across all test phrases, showing significantly better performance than other methods.
2. Pure Beam Decoding without language model consistently performs worst, with Levenshtein distances up to 22 characters from the target transcription.
3. Beam Decoding with LM Rescoring performs better than pure Beam Decoding but doesn't reach the accuracy of Greedy or Beam with integrated LM.
4. For some phrases (e.g., "AT THIS TIME ALL PARTICIPANTS..."), all methods struggle equally, suggesting that in challenging audio segments, even advanced language models offer limited improvement.
5. The perfect transcriptions (Levenshtein distance of 0) were achieved only by Greedy and Beam_LM decoding for one particular phrase, demonstrating the effectiveness of these methods.

Comparison Between 3-gram and 4-gram Language Models

Comparing the results between the 3-gram pruned model (3-gram.pruned.1e-7.arpa.gz) and the 4-gram model (4-gram.arpa.gz), we observe:

1. **No measurable performance difference:** The Levenshtein distances remained identical across all test phrases and decoding methods, suggesting that the higher-order n-gram model did not provide additional discriminative power for these specific test cases.
2. **Consistent pattern of effectiveness:** Both models show that Greedy Decoding and Beam Decoding with LM consistently outperform other methods, with identical distances for each test phrase.
3. **Efficiency implications:** Since the more complex 4-gram model did not improve results, the 3-gram model may be preferable for practical applications due to its smaller size and potentially faster inference.
4. **Domain specificity:** The consistency suggests that for this particular speech recognition domain and test set, the additional contextual information captured by 4-grams does not meaningfully impact transcription accuracy.

This finding is significant as it indicates that simply increasing the n-gram order may not always yield performance improvements in speech recognition tasks, particularly when the decoder's implementation balances acoustic and language model scores in specific ways.

Conclusions from Parameter Sweep Experiment

An experiment was conducted by varying **beam_width** (values: 2, 3, 5, 10), alpha (LM weight, values: 0.5, 1.0, 1.5, 2.0), and beta (word bonus, values: 0.5, 1.0, 1.5, 2.0) for the beam, **beam_lm**, and **beam_lm_rescore** decoding methods using a subset of the test audio samples. The **Levenshtein** distance was measured for each combination.

Key Observations:

1. **beam_lm Robustness and Superiority:** The **beam_lm** (Beam Search with LM shallow fusion) method consistently achieved the best results across *all tested parameter combinations*. Notably, its performance (**Levenshtein** distances of 0 for sample2.wav, 4 for sample4.wav, 15 for sample6.wav) was remarkably stable and largely insensitive to variations in **beam_width**, alpha, or beta within the tested ranges. This suggests the default parameters or the inherent structure of this method in the implementation are already near-optimal for this dataset.
2. **Impact of beam_width:**
 - For standard beam search, increasing **beam_width** from 2 or 3 to 5 or 10 provided noticeable improvements for sample4.wav (distance dropped from 19 to 14) and a slight improvement for sample2.wav (15 to 14).
 - For **beam_lm_rescore**, increasing **beam_width** also showed benefits, particularly for sample4.wav (distance dropped from 19 to 13 when moving from **beam_width** 3 to 5) and sample2.wav (11 to 10 when moving from 2 to 3). Increasing **beam_width** to 10 sometimes slightly worsened results for sample6.wav.
 - For **beam_lm**, **beam_width** variation had no impact on the already optimal results.
3. **Impact of alpha and beta:** Surprisingly, varying alpha (LM weight) and beta (word bonus) within the tested ranges had **minimal to no measurable impact** on the **Levenshtein** distance for both **beam_lm** and **beam_lm_rescore** methods in this experiment. The performance differences observed were almost entirely correlated with changes in **beam_width**. This could be due to the specific weighting factors used in the code, the dominance of the acoustic model score, or the specific characteristics of the test data and language model.
4. **Method Ranking:** The relative performance ranking remained consistent: **beam_lm** performed best, followed by **beam_lm_rescore**, with standard beam performing the worst. Parameter tuning slightly improved beam and **beam_lm_rescore**, but neither surpassed the consistent performance of **beam_lm**.

Overall Conclusion: The **beam_lm** method is the most effective and robust for this task and implementation, delivering the best results largely independent of parameter tuning within the explored ranges. While increasing **beam_width** can benefit **beam_lm_rescore** and standard beam search, tuning alpha and beta showed negligible effect in this specific setup. For optimal performance, **beam_lm** with a moderate **beam_width** (like the default 3, or potentially 5) is recommended.